



Projetos de Algoritmos e Técnicas de Programação

Prof. Dr. Paulino Wagner Palheta Viana

Manaus, Fevereiro 2026

Formato Básico do Pseudocódigo e Inclusão de Comentários

- O formato básico do nosso pseudocódigo é o seguinte:

```
algoritmo "seunome"  
// Função :  
// Autor :  
// Data :  
// Seção de Declarações  
inicio  
// Seção de Comandos  
finalgoritmo;
```

Tipos de Dados

- O VisuAlg prevê quatro tipos de dados: inteiro, real, cadeia de caracteres e lógico (ou booleano). As palavras-chave que os definem são as seguintes (observe que elas não tem acentuação):
 - inteiro: define variáveis numéricas do tipo inteiro, ou seja, sem casas decimais.
 - real: define variáveis numéricas do tipo real, ou seja, com casas decimais.
 - caractere: define variáveis do tipo string, ou seja, cadeia de caracteres.
 - logico: define variáveis do tipo booleano, ou seja, com valor VERDADEIRO ou FALSO.

Nomes de Variáveis e sua Declaração

- Os nomes das variáveis devem começar por uma letra e depois conter letras, números ou underline, até um limite de 30 caracteres. Não pode haver duas variáveis com o mesmo nome.
- A seção de declaração de variáveis começa com a palavra-chave `var`, e continua com as seguintes sintaxes:
 - `<lista-de-variáveis> : <tipo-de-dado>`
 - Na `<lista-de-variáveis>`, os nomes das variáveis estão separados por vírgulas.



Nomes de Variáveis e sua Declaração

■ Exemplo:

- ☐ var a: inteiro
- ☐ Valor1, Valor2: real
- ☐ nome_do_aluno: caractere
- ☐ sinalizador: logico

Constantes e Comando de Atribuição

- O VisuAlg tem três tipos de constantes:
 - •Numéricos: são valores numéricos escritos na forma usual das linguagens de programação. Podem ser inteiros ou reais. Neste último caso, o separador de decimais é o ponto e não a vírgula, independente da configuração regional do computador onde o VisuAlg está sendo executado. O VisuAlg também não suporta separadores de milhares.
 - Caracteres: qualquer cadeia de caracteres delimitada por aspas duplas (").
 - Lógicos: admite os valores VERDADEIRO ou FALSO.

Constantes e Comando de Atribuição

- A atribuição de valores a variáveis é feita com o operador `<-`. Do seu lado esquerdo fica a variável à qual está sendo atribuído o valor, e à sua direita pode-se colocar qualquer expressão (constantes, variáveis, expressões numéricas), desde que seu resultado tenha tipo igual ao da variável.
- Exemplos:
 - `a <- 3`
 - `Valor1 <- 1.5`
 - `Valor2 <- Valor1 + a`
 - `nome_do_aluno <- "José da Silva"`
 - `sinalizador <- FALSO`

Operadores

Operadores Aritméticos	
+, -	Operadores unários, isto é, são aplicados a um único operando. São os operadores aritméticos de maior precedência. Exemplos: -3, +x. Enquanto o operador unário - inverte o sinal do seu operando, o operador + não altera o valor em nada o seu valor.
\	Operador de divisão inteira. Por exemplo, $5 \setminus 2 = 2$. Tem a mesma precedência do operador de divisão tradicional.
+, -, *, /	Operadores aritméticos tradicionais de adição, subtração, multiplicação e divisão. Por convenção, * e / têm precedência sobre + e -. Para modificar a ordem de avaliação das operações, é necessário usar parênteses como em qualquer expressão aritmética.
MOD ou %	Operador de módulo (isto é, resto da divisão inteira). Por exemplo, $8 \text{ MOD } 3 = 2$. Tem a mesma precedência do operador de divisão tradicional.
^	Operador de potenciação. Por exemplo, $5 ^ 2 = 25$. Tem a maior precedência entre os operadores aritméticos binários (aqueles que têm dois operandos).

Operadores

Operadores de Caracteres

+	Operador de concatenação de strings (isto é, cadeias de caracteres), quando usado com dois valores (variáveis ou constantes) do tipo "caractere". Por exemplo: "Rio " + " de Janeiro" = "Rio de Janeiro".
---	---

Operadores Relacionais

=, <, >, <=, >=, <>	Respectivamente: igual, menor que, maior que, menor ou igual a, maior ou igual a, diferente de. São utilizados em expressões lógicas para se testar a relação entre dois valores do mesmo tipo. Exemplos: 3 = 3 (3 é igual a 3?) resulta em VERDADEIRO ; "A" > "B" ("A" está depois de "B" na ordem alfabética?) resulta em FALSO.
---------------------	---

Operadores

Operadores Lógicos	
nao	Operador unário de negação. não VERDADEIRO = FALSO, e não FALSO = VERDADEIRO. Tem a maior precedência entre os operadores lógicos.
ou	Operador que resulta VERDADEIRO quando um dos seus operandos lógicos for verdadeiro.
e	Operador que resulta VERDADEIRO somente se seus dois operandos lógicos forem verdadeiros.
xou	Operador que resulta VERDADEIRO se seus dois operandos lógicos forem diferentes, e FALSO se forem iguais.

Comandos de Saída de Dados

- Escreve no dispositivo de saída padrão (isto é, na área à direita da metade inferior da tela do VisuAlg) o conteúdo de cada uma das expressões que compõem <lista-de-expressões>. As expressões dentro desta lista devem estar separadas por vírgulas; depois de serem avaliadas, seus resultados são impressos na ordem indicada.

escreva (<lista-de-expressões>)

Comandos de Saída de Dados

- É possível especificar o número de espaços no qual se deseja escrever um determinado valor. Por exemplo, o comando `escreva(x:5)` escreve o valor da variável `x` em 5 espaços, alinhando-o à direita. Para variáveis reais, pode-se também especificar o número de casas fracionárias que serão exibidas. Por exemplo, considerando `y` como uma variável real, o comando `escreva(y:6:2)` escreve seu valor em 6 espaços colocando 2 casas decimais.

`escreval (<lista-de-expressões>)`

Idem ao anterior, com a única diferença que pula uma linha em seguida.

Comandos de Saída de Dados

■ Exemplos:

```
algoritmo "exemplo"
```

```
var x: real
```

```
y: inteiro
```

```
a: caractere
```

```
l: logico
```

```
inicio
```

```
x <- 2.5
```

```
y <- 6
```

```
a <- "teste"
```

```
l <- VERDADEIRO
```

```
escreval ("x", x:4:1, y+3:4) // Escreve: x 2.5 9
```

```
escreval (a, "ok") // Escreve: teste ok (e depois pula linha)
```

```
escreval (a, " ok") // Escreve: teste ok (e depois pula linha)
```

```
escreval (a + " ok") // Escreve: teste ok (e depois pula linha)
```

```
escreva (l) // Escreve: VERDADEIRO
```

```
fimalgoritmo
```

Comando de Entrada de Dados

- Recebe valores digitados pelos usuário, atribuindo-os às variáveis cujos nomes estão em <lista-de-variáveis> (é respeitada a ordem especificada nesta lista).

leia (<lista-de-variáveis>)

■ Exemplo

algoritmo "exemplo 1"

var x: inteiro;

inicio

leia (x)

escreva (x)

fimalgoritmo

Comando de Desvio Condicional

- Ao encontrar este comando, o VisuAlg analisa a <expressão-lógica>. Se o seu resultado for VERDADEIRO, todos os comandos da <sequência-de-comandos> (entre esta linha e a linha com fimse) são executados. Se o resultado for FALSO, estes comandos são desprezados e a execução do algoritmo continua a partir da primeira linha depois do fimse.

```
se <expressão-lógica> entao  
  <sequência-de-comandos>  
fimse
```

Comando de Desvio Condicional

- Nesta outra forma do comando, se o resultado da avaliação de <expressão-lógica> for VERDADEIRO, todos os comandos da <sequência-de-comandos-1> (entre esta linha e a linha com senao) são executados, e a execução continua depois a partir da primeira linha depois do fimse. Se o resultado for FALSO, estes comandos são desprezados e o algoritmo continua a ser executado a partir da primeira linha depois do senão.

```
se <expressão-lógica> entao  
<sequência-de-comandos-1>  
senao  
<sequência-de-comandos-2>  
fimse
```


Comando de Seleção Múltipla

- O VisuAlg implementa (com certas variações) o comando de seleção múltiplas. A sintaxe é a seguinte:

```
escolha <expressão-de-seleção>  
caso <exp11>, <exp12>, ..., <exp1n>  
<sequência-de-comandos-1>  
caso <exp21>, <exp22>, ..., <exp2n>  
<sequência-de-comandos-2>  
...  
outrocaso  
<sequência-de-comandos-extra>  
fimescolha
```

Comando de Seleção Múltipla

■ Exemplo:

algoritmo "Times"

var time: caractere

inicio

escreva ("Entre com o nome de um time de futebol: ")

leia (time)

escolha time

caso "Flamengo", "Fluminense", "Vasco", "Botafogo"

 escreval ("É um time carioca.")

caso "São Paulo", "Palmeiras", "Santos", "Corinthians"

 escreval ("É um time paulista.")

outrocaso

 escreval ("É de outro Estado.")

fimescolha

fimalgoritmo

Comandos de Repetição

- O VisuAlg implementa as três estruturas de repetição usuais nas linguagens de programação: o laço contado **para...ate...faca** (similar ao for do C), e os laços condicionados **enquanto...faca** (similar ao while) e **repita...ate** (similar ao do_while). A sintaxe destes comandos é explicada a seguir.

```
para <variável> de <valor-inicial> ate <valor-limite> [passo  
<incremento>] faca  
<sequência-de-comandos>  
fimpara
```

Comandos de Repetição

para...ate...faca

<variável >	É a variável contadora que controla o número de repetições do laço. Na versão atual, deve ser necessariamente uma variável do tipo inteiro, como todas as expressões deste comando.
<valor-inicial>	É uma expressão que especifica o valor de inicialização da variável contadora antes da primeira repetição do laço.
<valor-limite >	É uma expressão que especifica o valor máximo que a variável contadora pode alcançar.
<incremento >	É opcional. Quando presente, precedida pela palavra passo, é uma expressão que especifica o incremento que será acrescentado à variável contadora em cada repetição do laço. Quando esta opção não é utilizada, o valor padrão de <incremento> é 1. Vale a pena ter em conta que também é possível especificar valores negativos para <incremento>. Por outro lado, se a avaliação da expressão <incremento > resultar em valor nulo, a execução do algoritmo será interrompida, com a impressão de uma mensagem de erro.
fimpara	Indica o fim da sequência de comandos a serem repetidos. Cada vez que o programa chega neste ponto, é acrescentado à variável contadora o valor de <incremento >, e comparado a <valor-limite >. Se for menor ou igual (ou maior ou igual, quando <incremento > for negativo), a sequência de comandos será executada mais uma vez; caso contrário, a execução prosseguirá a partir do primeiro comando que esteja após o fimpara.

Comandos de Repetição

■ Exemplo:

algoritmo "Números de 1 a 10"

var j: inteiro

inicio

para j de 1 ate 10 faca

escreva (j:3)

fimpara

fimalgoritmo

Comandos de Repetição

- Exemplo de decremento utilizando passo -1:

```
algoritmo "Numeros de 10 a 1 (este funciona)"
```

```
var j: inteiro
```

```
inicio
```

```
para j de 10 ate 1 passo -1 faca
```

```
  escreva (j:3)
```

```
fimpara
```

```
fimalgoritmo
```

Comandos de Repetição

- O **enquanto...faca**. Esta estrutura repete uma sequência de comandos enquanto uma determinada condição (especificada através de uma expressão lógica) for satisfeita.

enquanto <expressão-lógica> faca

<sequência-de-comandos>

fimenquanto

Enquanto ... faca

<expressão-lógica>	Esta expressão que é avaliada antes de cada repetição do laço. Quando seu resultado for VERDADEIRO, <sequência-de-comandos> é executada.
fimenquanto	Indica o fim da <sequência-de-comandos> que será repetida. Cada vez que a execução atinge este ponto, volta-se ao início do laço para que <expressão-lógica> seja avaliada novamente. Se o resultado desta avaliação for VERDADEIRO, a <sequência-de-comandos> será executada mais uma vez; caso contrário, a execução prosseguirá a partir do primeiro comando após fimenquanto.

Comandos de Repetição

■ Exemplo:

```
algoritmo "Números de 1 a 10 (com enquanto...faca)"  
var j: inteiro  
inicio  
j <- 1  
enquanto j <= 10 faca  
  escreva (j:3)  
  j <- j + 1  
fimenquanto  
fimalgoritmo
```


Comandos de Repetição

- O **repita...até**. Esta estrutura repete uma sequência de comandos até que uma determinada condição (especificada através de uma expressão lógica) seja satisfeita.

repita

<sequência-de-comandos>

ate <expressão-lógica>

Repita ...ate

repita	Indica o início do laço.
ate <expressão-lógica>	Indica o fim da <sequência-de-comandos> a serem repetidos. Cada vez que o programa chega neste ponto, <expressão-lógica> é avaliada: se seu resultado for FALSO, os comandos presentes entre esta linha e a linha repita são executados; caso contrário, a execução prosseguirá a partir do primeiro comando após esta linha.

Comandos de Repetição

■ Exemplo:

```
algoritmo "Números de 1 a 10 (com repita)"  
var j: inteiro  
inicio  
  j <- 1  
  repita  
    escreva (j:3)  
    j <- j + 1  
  ate j > 10  
finalgoritmo
```



Projetos de Algoritmos e Técnicas de Programação

Prof. Dr. Paulino Wagner Palheta Viana

Manaus, Fevereiro 2026